# Frontend & Panel Security Review

# Frontend & Panel Security Review

**Scope:** The React single-page application in `client/src` together with the build/deploy pipeline that ships it ( `client` , `.github/workflows/deploy-frontend.yml` ). This covers credentials baked into the bundle, tokens and PII kept in the browser, HTML-injection sinks, route gating and response headers.

**Date:** 2026-10-09 **Method:** Read the source and grep for known-dangerous patterns ( `dangerouslySetInnerHTML` , `innerHTML` , `document.write` , `localStorage` , `VITE_*` secrets). Every finding cites the file and line that proves it.

**Related:** server-side counterparts of these issues are in BACKEND-SECURITY.md and DATABASE-SECURITY.md. The frontend is a *presentation* layer — anything enforced only here can be bypassed by calling the API directly. It is still worth fixing, because a bundle is text (obfuscation is not protection) and because XSS in the bundle steals whatever the browser holds.

---

## 1. Summary

| ID | Severity | Finding | Status |
|---|---|---|---|
| FR-001 | **Critical** | Shared “ticket gate” password is hardcoded in the JS bundle | Open |
| FR-002 | **High** | Untrusted HTML rendered without sanitisation (XSS sinks, no DOMPurify) | Open |
| FR-003 | **High** | WhatsApp master credentials injected into the frontend build | Open |
| FR-004 | **High** | Session token and role stored in `localStorage` , trusted client-side | Open |
| FR-005 | **Medium** | Impersonation keeps the original user’s token in `localStorage` | Open |
| FR-006 | **Medium** | Donor PII cached in `localStorage` by the Accounts dashboard | Open |
| FR-007 | **Medium** | Internal documentation panel open to every authenticated role | Open |
| FR-008 | **Medium** | Unescaped values written into a new window via `document.write` | Open |
| FR-009 | Low | No CSP / security headers on the client host | Open |
| FR-010 | Info | Verbose renderer error injected into the DOM | Open |

Fix order: **FR-001** → **FR-003** → **FR-002** → **FR-004** → **FR-007** → **FR-005/FR-006** → **FR-008**.

---

## 2. Findings

### FR-001 — Hardcoded “ticket gate” credentials in the bundle

**Severity:** Critical · Confirmed

```
// client/src/components/TicketGate.jsx:4-8
const TICKET_GATE = {
  username: 'jatinsevak@ufs',
  password: 'sevak123',
  sessionKey: 'ucs_ticket_unlocked',
};
```

```
// client/src/components/TechnicalTickets.jsx:8-12
const TICKET_GATE = {
  username: 'jatinsevak@ufs',
  password: 'sevak123',
  sessionKey: 'ucs_ticket_unlocked',
};
```

The same credential is duplicated in two components and compared only in the browser ( `TicketGate.jsx:32` ). It is compiled into the production JavaScript and is visible to anyone who opens DevTools → Sources or fetches the bundle. The unlock is a `sessionStorage` flag ( `ucs_ticket_unlocked` ), so no server check ever happens. This is obfuscation, not access control.

**Remediation** - Nothing secret belongs in client code. Delete the gate or replace it with a real server-side authorisation check on the ticket API (see BE-001 — the endpoint currently accepts *any* authenticated role). - Treat `sevak123` / `jatinsevak@ufs` as compromised and rotate it everywhere it is reused.

---

### FR-002 — Untrusted HTML rendered without sanitisation

**Severity:** High · Confirmed

React escapes interpolated values by default; these five sites deliberately opt out and none of them sanitise first. A search of `client/src` finds **no** `DOMPurify` and no sanitiser dependency at all — only `mammoth` (DOCX → HTML) is present.

| Sink | Source of HTML |
|---|---|
| `client/src/panels/hr/components/Letters.jsx:1166` ( `el.innerHTML = bodyText` ) | generated letter body |
| `client/src/panels/hr/components/Letters.jsx:1599` ( `dangerouslySetInnerHTML={{ __html: out.body }}` ) | generated letter body |
| `client/src/panels/accounts/pages/Certificates.jsx:80` ( `dangerouslySetInnerHTML={{ __html: html }}` ) | `mammoth.convertToHtml` of a **user-uploaded** `.docx` ( `Certificates.jsx:56` , `:453` ) |
| `client/src/panels/documentation/components/DiagramViewer.jsx:67` ( `container.innerHTML = ... ${e.message}` ) | mermaid renderer error message |
| `client/src/panels/sevak-library/MembershipFormDoc.jsx:156` ( `node.innerHTML = a4Html(rows[i], ...)` ) | form field values |

The Certificates case is the worst: the HTML comes from a file the user uploads, so a crafted `.docx` containing `<img src=x onerror=...>` becomes script in the admin's session. That session holds the token described in FR-004, so this is a direct path from "upload a template" to "exfiltrate an admin token".

**Remediation** - Add `dompurify` and wrap every one of these sinks: `dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }}` . - For mermaid, prefer rendering to an SVG string via the library's own sanitiser option and keep `securityLevel: 'strict'` (verify the mermaid call sets this). - Prefer structured React elements over HTML strings wherever the markdown/HTML conversion is not genuinely required. - Add an ESLint rule ( `react/no-danger` ) so new sinks need a review.

---

## FR-003 — WhatsApp master credentials injected into the frontend build

**Severity:** High · Confirmed

```
# .github/workflows/deploy-frontend.yml:43-44
VITE_WHATSAPP_MASTER_EMAIL: ${{ secrets.CRM_WHATSAPP_EMAIL }}
VITE_WHATSAPP_MASTER_PASSWORD: ${{ secrets.CRM_WHATSAPP_PASSWORD }}
```

Every `VITE_*` variable is **inlined into the client bundle at build time** and is readable in the shipped JavaScript. These two lines bake the CRM's WhatsApp master login into the public app. The values are not referenced anywhere in `client/src` today (grep returns nothing), so the switch is dead — but the credentials are still exported into the build artefacts on every deploy.

**Remediation** - Remove these two `VITE_*` lines from the workflow. The client must never hold a master credential. - If WhatsApp automation is needed, do it on the backend and expose a scoped, authenticated endpoint. - Rotate the WhatsApp master email/password (assume they leaked), and audit every `VITE_*` value in the workflow — anything secret is already public.

---

## FR-004 — Session token and role in `localStorage`, trusted client-side

**Severity:** High · Confirmed

```
// client/src/store.jsx:57-59
const [user, setUser] = useState(() => getUser('ucs'))
const [token, setToken] = useState(() => getToken('ucs'))
```

```
// client/src/App.jsx:56-66
function ProtectedRoute({ role, children }) {
  const { user } = useUcs()
  ...
  if (!allowedRoles.includes(user.role) && !allowedRoles.includes(user.department)) {
    return <AccessDenied />
  }
}
```

`setSession` ( `client/src/api/auth.js:5-22` ) writes the JWT and the whole user object to `localStorage` . The route guard then reads `user.role` *from that same localStorage*. A user can edit `ucs_user` in DevTools and any role gate in `App.jsx` ( `/super-admin` , `/hr` , `/accounts` , ...) opens immediately. This is only a cosmetic bypass **as long as every API endpoint re-checks the role server-side** — which BE-001 shows is not the case for developer tickets.

`localStorage` is also readable by any JavaScript on the origin, so FR-002 XSS converts straight into token theft. The token is not `HttpOnly` .

**Remediation** - Never trust client-stored role for authorisation; keep the server as the source of truth (fix BE-001 and audit every route). - Move the session token to an `HttpOnly; Secure; SameSite` cookie so script cannot read it, and stop persisting the user/role object client-side. - Fix FR-002 so XSS cannot read the token while it is still in storage.

---

### FR-005 — Impersonation keeps the original user's token

**Severity:** Medium · Confirmed

```
// client/src/api/auth.js:169-185 (startWorkAs)
localStorage.setItem('ucs_orig_token', ...)
localStorage.setItem('ucs_orig_user', ...)
```

The "work as / impersonate" flow stashes the operator's real token and user object in `localStorage` under `ucs_orig_*` so it can be restored later. Those keys survive for the whole impersonation session and are readable by any script on the page (FR-002). They are also not obviously cleared by the logout path, which only allow-lists three unrelated prefix groups ( `store.jsx:84-95` ).

**Remediation** - Keep the operator token in memory only, or in a short-lived server-side session; do not persist it in `localStorage` . - Ensure logout and impersonation-end clear `ucs_orig_*` explicitly, and add them to a test.

---

### FR-006 — Donor PII cached in `localStorage`

**Severity:** Medium · Confirmed

```
// client/src/panels/accounts/pages/Dashboard.jsx:224-225
localStorage.setItem('receipts_verified_data', JSON.stringify(rows));
localStorage.setItem('receipts_verified_count', String(verified.length));
```

`rows` are receipt records that include donor name, mobile and `pan_number` (see the field list in `AddressImport.jsx:10` ). The whole set is serialised to `localStorage` , where it stays readable — including to the XSS sinks in FR-002 — and can be dumped wholesale with a one-line snippet. It is cleared on a clean logout but not on a closed tab or a crash.

**Remediation** - Do not persist raw PII to `localStorage` . Keep it in memory / a React store, or persist only non-sensitive identifiers and re-fetch from the API. - If caching is required, store it server-side (session) and send only what the current view needs.

---

### FR-007 — Internal documentation panel open to every authenticated role

**Severity:** Medium · Confirmed

```
// client/src/App.jsx:227-232
<Route path="/docs/*" element={
  <ProtectedRoute role={['*']}>
    ...
    <DocumentationPanel />
```

`ProtectedRoute` returns `children` unconditionally when `'*'` is in the allowed list ( `App.jsx:60` ). Combined with `DocumentationPanel` 's lazy import ( `App.jsx:20` ), the internal architecture/runbook docs are reachable by **any logged-in user of any role** at `/docs/*` .

**Remediation** - Restrict the docs route to the roles that need it (e.g. `super_admin` , `developers` ) on the server *and* in the router. - If the docs reveal infrastructure detail, treat them as privileged content and do not ship them in the public bundle at all.

---

### FR-008 — Unescaped values written into a new window via `document.write`

**Severity:** Medium · Confirmed

```
// client/src/panels/hr/components/GenerateQR.jsx:76-79
const win = window.open('', '_blank');
```

```
...
win.document.write(`<h2>${label}</h2>`);
```

```
// client/src/panels/hr/components/forms/PrintForms.jsx:22-24
const content = ref.current.innerHTML
printWindow.document.write(`...${content}...`)
```

`label` is not escaped before being written into the popup's HTML, and the PrintForms popup copies the current DOM's `innerHTML` verbatim. The popup is same-origin ( `window.open('', '_blank')` ), so script executing there runs in the app's origin and can reach the opener's `localStorage` /session.

**Remediation** - Build the popup DOM with `createElement` / `textContent` , or escape/allow-list before `document.write` . - Prefer a hidden in-app print view over popup + `document.write` , and run any copied markup through the FR-002 sanitiser.

---

### FR-009 — No CSP / security headers on the client host

**Severity:** Low · Confirmed

`client/index.html` contains no `<meta http-equiv="Content-Security-Policy">` , and the repository has no `_headers` , `vercel.json` or equivalent that would set a CSP, `X-Frame-Options` / `frame-ancestors` , or `Referrer-Policy` for the static site. (The API-side gap is BE-002.) A restrictive CSP would not stop FR-001 but would sharply reduce the impact of FR-002/FR-008.

**Remediation** - Add a CSP at the static host: at minimum `default-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self'` . Budget for the inline styles/host the app uses, and tighten over time. - Add `Strict-Transport-Security` , `X-Content-Type-Options: nosniff` and `Referrer-Policy: strict-origin-when-cross-origin` .

---

### FR-010 — Verbose renderer error injected into the DOM

**Severity:** Info · Confirmed

```
// client/src/panels/documentation/components/DiagramViewer.jsx:67
container.innerHTML = `<pre ...>Diagram render error: ${e.message}</pre>`
```

Raw exception messages are shown to the user and (combined with the missing sanitiser) interpolated into HTML. Low on its own; it is a second example of the FR-002 pattern and can leak internal detail.

**Remediation** - Render the message as text ( `textContent` ), and show a generic message to the user while logging detail to the console.

---

## 3. Action plan

1. **FR-001** — remove the in-bundle credential and rotate `sevak123` / `jatinsevak@ufs` ; gate tickets on the server (pairs with BE-001).
2. **FR-003** — delete the two `VITE_WHATSAPP_MASTER_*` lines from the deploy workflow and rotate the WhatsApp master secret.
3. **FR-002** — add `dompurify` and sanitise all five sinks; enable `react/no-danger` .
4. **FR-004** — stop trusting client-stored role; move the token to an `HttpOnly` cookie; audit server-side role checks.
5. **FR-007** — restrict `/docs/*` to privileged roles.
6. **FR-005 / FR-006** — stop persisting `ucs_orig_*` and donor PII in `localStorage` .
7. **FR-008** — escape/stop `document.write` in the QR and print popups.
8. **FR-009** — add CSP and security headers at the host.

## 4. Verification

- `grep -rn "jatinsevak@ufs\|sevak123" client/src` returns nothing after FR-001.
- `grep -rn "dangerouslySetInnerHTML\|\.innerHTML\s*=" client/src` shows each result wrapped in `DOMPurify.sanitize(...)` (or removed) after FR-002.
- `.github/workflows/deploy-frontend.yml` has no secret `VITE_*` line, and a production bundle grep for the WhatsApp master email returns nothing (FR-003).
- DevTools → Application → Local Storage contains no `ucs_token`, `ucs_orig_*` or `receipts_verified_data` after FR-004/005/006.
- A crafted `xss.docx` (containing `<img src=x onerror=alert(1)>`) uploaded to Certificates no longer executes after FR-002.
- `/docs/*` returns an access-denied view for a non-privileged role after FR-007.